Pointers to function (Function Pointer)

A function pointer is a pointer variable that stores the address of a function. Using this pointer, we can call the function.

return_type (*pointer_name)(parameter_types);

Function pointer without arguments

```
#include <stdio.h>
void display() {
    printf("Hello Students!");
}

int main() {
    void (*ptr)();
    ptr = display;
    ptr();
    return 0;
}
```

Hello Students!

Function pointer with arguments and return type

```
#include <stdio.h>
int add(int a, int b) {
    return a + b;
}

int main() {
    int (*ptr)(int, int);
    ptr = add;
    printf("%d", ptr(10, 20));
    return 0;
}
```

30

Function pointer with two functions

```
#include <stdio.h>
int add(int a, int b) {
```

```
    return a + b;
}
int multiply(int a, int b) {
    return a * b;
}

int main() {
    int (*ptr)(int, int) = add;
    printf("Addition = %d\n", ptr(5, 3));
    ptr = multiply;
    printf("Multiplication = %d\n", ptr(5, 3));
    return 0;
}
```

Addition = 8

Multiplication = 15

Function Pointer Array

```
#include <stdio.h>
void add() { printf("Add\n"); }
void subtract() { printf("Subtract\n"); }
void multiply() { printf("Multiply\n"); }

int main() {
  void (*ptr[3])() = { add, subtract, multiply };
  for (int i = 0; i < 3; i++) {
    ptr[i]();
  }
  return 0;
}
```

Add

Subtract

Multiply

```
#include <stdio.h>
void add(int a, int b) {
```

```
    printf("Result: %d\n", a + b);
}
void subtract(int a, int b) {
    printf("Result: %d\n", a - b);
}
void multiply(int a, int b) {
    printf("Result: %d\n", a * b);
}

int main() {
    void (*ptr[3])(int, int) = {add, subtract, multiply};
    int x = 10, y = 5;
    for (int i = 0; i < 3; i++) {
        ptr[i](x, y);
    }
    return 0;
}
```

Result: 15

Result: 5

Result: 50